



佛山大学
FOSHAN UNIVERSITY



计算机与人工智能学院
School of Computer Science and Artificial Intelligence

Section 6

NLP—文本向量化

Natural Language Processing — Text Vectorization

任课教师：陈荟慧

目录 CONTENTS



01

文本向量化简介

02

文本离散表示

03

分布式表示

04

论文相似度计算



文本向量化概述

- 文本向量化是将文本表示成一系列能够表达文本语义的机读向量。





文本向量化概述

- 文本向量化按照**向量化的粒度**可以将其分为以字单位、以词为单位和以句子为单位向量表达，根据不同的情景选择不同的向量表达方式和处理方式。
- 随着深度学习技术的广泛应用，**基于神经网络的文本向量化已经成为NLP领域的研究热点**，尤其是以词为单位的向量化。





文本向量化概述

- Word2Vec是目前以词为单位中最典型的生成词向量的工具，其特点是将所有的词向量化，这样词与词之间即可度量它们之间的关系、挖掘词之间的联系。
- 也有一部分研究将句子作为文本处理的基本单元，于是就产生了Doc2Vec和Str2Vec等技术。



目录 CONTENTS



01

文本向量化简介

02

文本离散表示

03

分布式表示

04

论文相似度计算



词向量表示

- 文本向量化主要有**离散表示**和**分布式表示**。





离散表示

- 离散表示将文本中的每个单词或短语映射到一个固定的、离散的向量空间中，通常基于词汇表来构建。每个单词对应向量空间中的一个维度，向量的元素通常为 0 或 1，表示该单词是否在文本中出现，或者也可以是单词的出现频率等其他统计信息。
- 离散表示是一种**基于规则**和**统计**的向量化方式，常用的方法有词集模型和词袋 (BOW) 模型，两类模型都是基于词之间保持独立性、没有关联为前提，将所有文本中的词形成一个字典，然后根据字典统计词出现频数。





One-hot表示

- 词集模型中的独热表示 (one-hot representation) , 只要单个文本中单词出现在字典中, 就将其置为1, 不管出现多少次。
- 这是一种最简单的离散表示方法。假设词汇表中有 n 个单词, 对于文本中的每个单词, 将其表示为一个长度为 n 的向量, 其中只有对应单词在词汇表中的位置为1, 其余位置均为0。
- 例如, 对于词汇表 ["苹果", "香蕉", "橙子"], 词“香蕉”的独热编码为 [0, 1, 0]。
- 例如, 粉凤凰, 红凤凰, 粉红凤凰花凤凰, One-hot表示为?





One-hot表示

- 用一个长的向量表示一个词，向量长度为词典的大小，每个向量只有一个维度为1，其余维度全部为0，为1的位置表示该词语在词典的位置。
- One-hot表示的缺点。
 - **维数过高**：随着语料的增加，维数会越来越大，导致维数灾难。
 - **矩阵稀疏**：one-hot表示的每一个词向量只有1维是有数值的，其他维度上的数值都为0。
 - **不能保留语义**：one-hot表示的结果不能保留词语在句子中的位置信息。





BOW表示

- 在独热编码的基础上，词袋模型不仅考虑单词是否出现，还统计单词在文本中出现的次数。例如，对于文本“我喜欢苹果，我也喜欢橙子”，基于上述词汇表，其词袋模型表示为 $[1, 0, 1]$ ，如果考虑词频，则表示为 $[2, 0, 1]$ 。
- BOW模型只要文本中一个词出现在字典中，就将其向量值加1，出现多少次就加多少次。





BOW模型

- BOW模型就是用一个向量表示一句话或一个文档。
- BOW模型忽略文档的词语顺序、语法、句法等要素，将文档看作是若干个词汇的集合，文档中每个词都是独立的。
- BOW模型每个维度上的数值代表ID对应的词在句子里出现的频次。





BOW模型与one-hot存在的缺点

- BOW模型与one-hot存在的缺点如下：
 - **维数高和稀疏性**：不能保留词语在句子中的位置信息，如“我为你鼓掌”和“你为我鼓掌”向量化结果依然没有区别。
 - **不能保留语义**：当语料增加时，维数也会增大，一个文本里不出现的词就会增多，导致矩阵稀疏。





文本离散表示的特点和问题

- 文本离散表示的特点是忽略文本信息中的语序信息和语境信息，仅将其反映为若干维度的独立概念。
- 这类模型由于本身原因存在无法解决的问题，如主语和宾语的顺序问题，会导致无法理解诸如“我想你”和“你想我”两个语句之间的区别。
- 例如，老王是谁的儿子？老王的儿子是谁？





TF-IDF表示

- **TF-IDF表示是用一个向量表示一个句话或一个文档**，它的表示过程如下。
 - 首先TF-IDF是在BOW的基础上对词出现的频次赋予TF-IDF权值。
 - 其次对BOW模型进行修正，进而表示该词在文档集合中的重要程度。


$$\frac{A}{r} = \pi r^2$$



离散化表示的特点

- 优点：简单直观，易于理解和实现，计算量相对较小，在一些简单的文本处理任务中，如文本分类的早期阶段，可能会有一定的效果。同时，离散表示的向量具有明确的物理意义，每个维度对应一个单词，可解释性强。
- 缺点：存在**维度灾难问题**，当词汇表规模较大时，向量维度会非常高，导致**数据稀疏**和计算资源的浪费。而且，这种表示方式**忽略了单词之间的语义关系**和文本的上下文信息，无法捕捉单词的相似性和文本的语义结构。



目录 CONTENTS



01

文本向量化简介

02

文本离散表示

03

分布式表示

04

论文相似度计算



分布式表示

- 分布式表示将文本中的单词或短语表示为低维向量空间中的向量，通过学习单词在大量文本中的上下文信息来捕捉其语义和句法特征。这种表示方式认为，在相似上下文中出现的单词具有相似的语义，因此它们在向量空间中的表示也应该相近。





分布式表示常见方法

- **Word Embedding**: 这是一种典型的分布式表示方法，**通过神经网络等模型在大规模语料上进行训练，学习到每个单词的低维向量表示**。例如，Word2Vec、GloVe 等模型都是常用的 Word Embedding 方法。以 Word2Vec 为例，它通过预测单词的上下文或根据上下文生成单词来学习词向量，使得语义相似的单词在向量空间中距离较近。
- **基于深度学习的句子或文档表示**: 如使用循环神经网络（RNN）、长短时记忆网络（LSTM）、卷积神经网络（CNN）等深度学习模型对句子或文档进行编码，将整个句子或文档表示为一个低维向量。这些模型可以捕捉文本中的长序列信息和语义结构，从而得到更准确的文本表示。





大模型知识库中常用的 Word Embedding 方法

- 常见的有以下几种：
- Word2Vec：是一种基于神经网络的模型，包括连续词袋模型（CBOW）和 Skip-gram 模型。CBOW 通过上下文预测中心单词，Skip-gram 则通过中心单词预测上下文单词，利用这些预测任务训练出来的神经网络权重作为单词的嵌入。
- GloVe：全称为 Global Vectors for Word Representation，是由斯坦福大学自然语言处理小组开发的一个开源项目，是一种基于共现矩阵的模型。该模型使用统计方法计算单词之间的关联性，然后通过奇异值分解（SVD）来生成嵌入。它在计算上比 Word2Vec 更快，并且可以扩展到更大的数据集。





大模型知识库中常用的 Word Embedding 方法

- FastText: 是由 Facebook 人工智能研究实验室 (FAIR) 开发的。在 Word2Vec 的基础上添加了字符级别的 n - gram 特征。这使得 FastText 可以将未知单词的嵌入表示为已知字符级别 n - gram 特征的平均值, 在处理不规则单词和罕见单词时表现出色。
- ELMO (Embedding from Language Model) : 由华盛顿大学开发, 是一种基于循环神经网络 (RNN) 的上下文相关的词嵌入方法。它通过在大规模语料上训练语言模型, 学习到每个单词在不同上下文语境下的向量表示, 能够很好地处理一词多义等问题。





大模型知识库中常用的 Word Embedding 方法

- BERT (Bidirectional Encoder Representations from Transformers) : 2018 年, 谷歌发布了 BERT 模型, BERT 是基于 Transformer 的预训练模型, 虽然不是专门的 Word Embedding 方法, 但可以生成句子或短文档的向量表示, 也能用于获取词向量。它通过自注意力机制捕捉文本中的长距离依赖关系和语义信息, 在各种自然语言处理任务中取得了显著的成果。
- 此外, 一些大模型还可能会使用 OpenAI 的 Embeddings 等通过 API 接口获取的词嵌入表示。





OpenAI 的 Embeddings

- 是一种将文本（如单词、句子或文档）转换为向量表示的技术。
- 特点：
 - 语义捕捉能力强：基于大规模语言模型训练，能够理解上下文和复杂语义，捕捉文本的语义信息，包括单词、句子甚至文档的含义。
 - 多语言支持：支持多种语言的文本嵌入，如英语、中文、西班牙语等。
 - 高维向量表示：生成的向量通常是高维的，如 1536 维或 3072 维，能够表示丰富的语义信息。
 - 易于使用：通过 OpenAI API 可以轻松获取文本的嵌入向量，便于开发者集成到自己的应用程序中。





OpenAI 的 Embeddings

- 工作原理：**基于 Transformer 架构的模型（如 GPT 模型）**，首先接收一段文本，将其转换为模型可理解的 token 序列，然后通过模型的隐藏层生成文本的嵌入向量，最后返回一个高维向量，表示文本的语义信息。
- 嵌入模型：OpenAI 提供了多个嵌入模型，如 text - embedding - 3 - large、text - embedding - 3 - small、text - embedding - ada - 002 等。不同模型在输出维度、计算效率和适用任务上有所不同。
- 例如，text - embedding - 3 - large 适合复杂的语义分析任务，text - embedding - 3 - small 适合简单的关键词搜索和快速文本分类，text - embedding - ada - 002 则在多种任务中表现较为平衡，适用于内容推荐和一般文本分析等任务。





分布式表示

- 分布式表示将每个词根据上下文从高维空间映射到一个低维度、稠密的向量上。
- 分布式表示的思想是词的语义是**通过上下文信息**确定的，即相同语境出现的词，其语义也相近。
- 分布式表示的优点是考虑到了词之间存在的相似关系，减小了词向量的维度。
- 常用的方法有基于矩阵的分布表示、基于聚类的分布表示和基于神经网络的分布表示。
- 分布式表示与one-hot表示对比，在形式上，one-hot表示的词向量是一种稀疏词向量，其长度就是字典长度，而分布式表示是一种固定长度的稠密词向量。





Word2Vec模型

- Word2Vec模型是简单化的神经网络模型。Word2Vec可以在百万数量级的词典和上亿的数据集上进行高效地训练；该工具得到的训练结果可以很好地度量词与词之间的相似性。





Word2Vec模型

- 模型特点是，当模型训练好后，并不会使用训练好的模型处理新的任务，真正需要的是模型通过训练数据所学得的参数，如隐藏层的权重矩阵。





Word2Vec模型

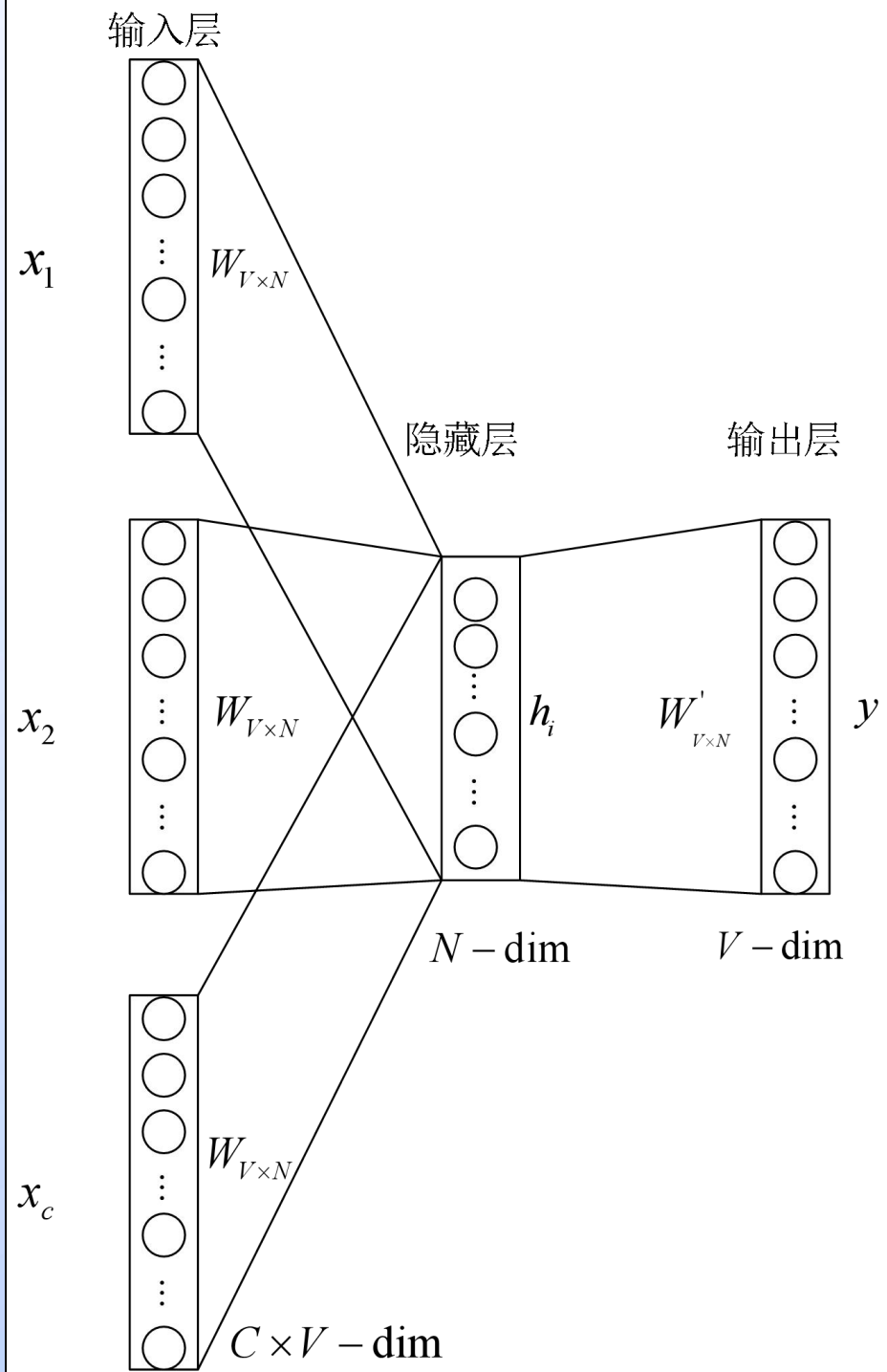
- Word2Vec模型的输入是one-hot向量，根据输入和输出模式不同，分为连续词袋模型（Continuous Bag-of-Words, CBOW）和跳字模型（Skip-Gram）。



Word2Vec模型

• 1. CBOW模型

- CBOW模型是根据上下文的词语预测目标词出现的概率。
- CBOW模型的神经网络包含了输入层、隐藏层和输出层。
 - 输入层的输入的是某一个特定词上下文的one-hot向量。
 - 输出层的输出是在给定上下文的条件下特定词的概率分布。
- 其模型结构如图所示。





Word2Vec模型

• (1)CBOW模型的网络结构

- 假设某个特定词的上下文含C个词，词汇表中词汇量的大小为V，每个词都用one-hot向量表示，神经网络相邻层的神经元是全连接的。其网络结构如下。
 - 输入层的含有C个单元，每个单元含V个神经元，用于输入V维one-hot向量。
 - 隐藏层的神经元个数为 N ，在输入层中，每个单元到隐藏层连接权重值共享一个_____维的权重矩阵 。
 - 输出层含有V个神经元，隐藏层到输出层连接权重为_____维权重矩阵 。
 - 输出层神经元的输出值表示词汇表中每个词的概率分布，通过softmax函数计算每个词出现的概率。

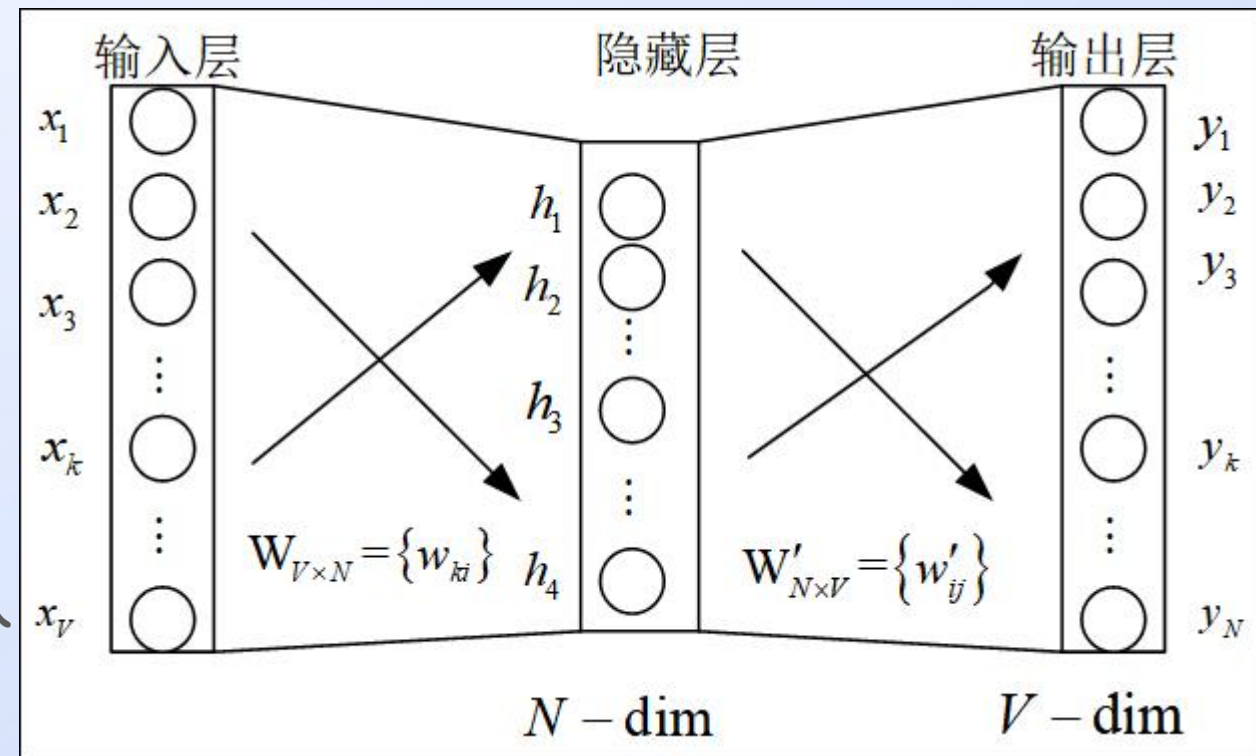


Word2Vec模型

• (2)CBOW模型的数字形式

• CBOW模型的简单形式

- 假定预测目标值只有一个词，模型是在只有一个上下文的情况下预测一个目标词。
- 此时CBOW模型的结构如右图所示。
- 模型的输入为一个V维one-hot词向量，输出也是一个V维向量。
- 它包含了V个词的概率，每一个概率代表输入一个词的条件下输出词的概率。





Word2Vec模型

• CBOW模型的一般形式

- 假设给定了一个输入 x (one-hot向量)，则隐藏层输出表达式可表示为_____。输出层每个神经元的输入表达式为_____。其中_____是矩阵_____的第 j 列向量。

- 填空

$$u_j = w_j'^T \cdot h \quad w_j' \quad W' \quad h = x^T W$$

- 词汇表中的词用字母 w 表示， w_I 表示实际输入的词， w_O 表示目标词，词汇表中第 j 个词用 w_j 表示，条件概率 $p(w_O = w_j | w_I)$ 表示输入词为 w_I ，目标词为 w_j 的概率。
- 当输入词为 w_I ，词汇表中每个词为目标词的概率式如下。

$$y_j = p(w_O = w_j | w_I) = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})}, j = 1, 2, \dots, V$$





Word2Vec模型

• (3)CBOW模型的损失函数

- CBOW模型输出的真实目标词的概率记为 $p(w_O|w_{I_1}, w_{I_2}, \dots, w_{I_C})$ ，训练的目的就是使这个概率最大，其表达式如下。

$$\begin{aligned} \max p(w_O|w_{I_1}, w_{I_2}, \dots, w_{I_C}) &= \max \log p(w_O|w_{I_1}, w_{I_2}, \dots, w_{I_C}) \downarrow \\ &= \max \log \frac{\exp(u_{j^*})}{\sum_{j'=1}^V \exp(u_{j'})} \downarrow \\ &= \max(u_{j^*} - \log \sum_{j'=1}^V \exp(u_{j'})) \end{aligned}$$

- 其中 j^* 表示真实目标词在词汇表中的下标。在求解过程中，要求目标函数的最小值，因此，损失函数的表达式定义如下。

$$E = -\log p(w_O|w_{I_1}, w_{I_2}, \dots, w_{I_C}) = \log \sum_{j'=1}^V \exp(u_{j'}) - u_{j^*}$$





Word2Vec模型

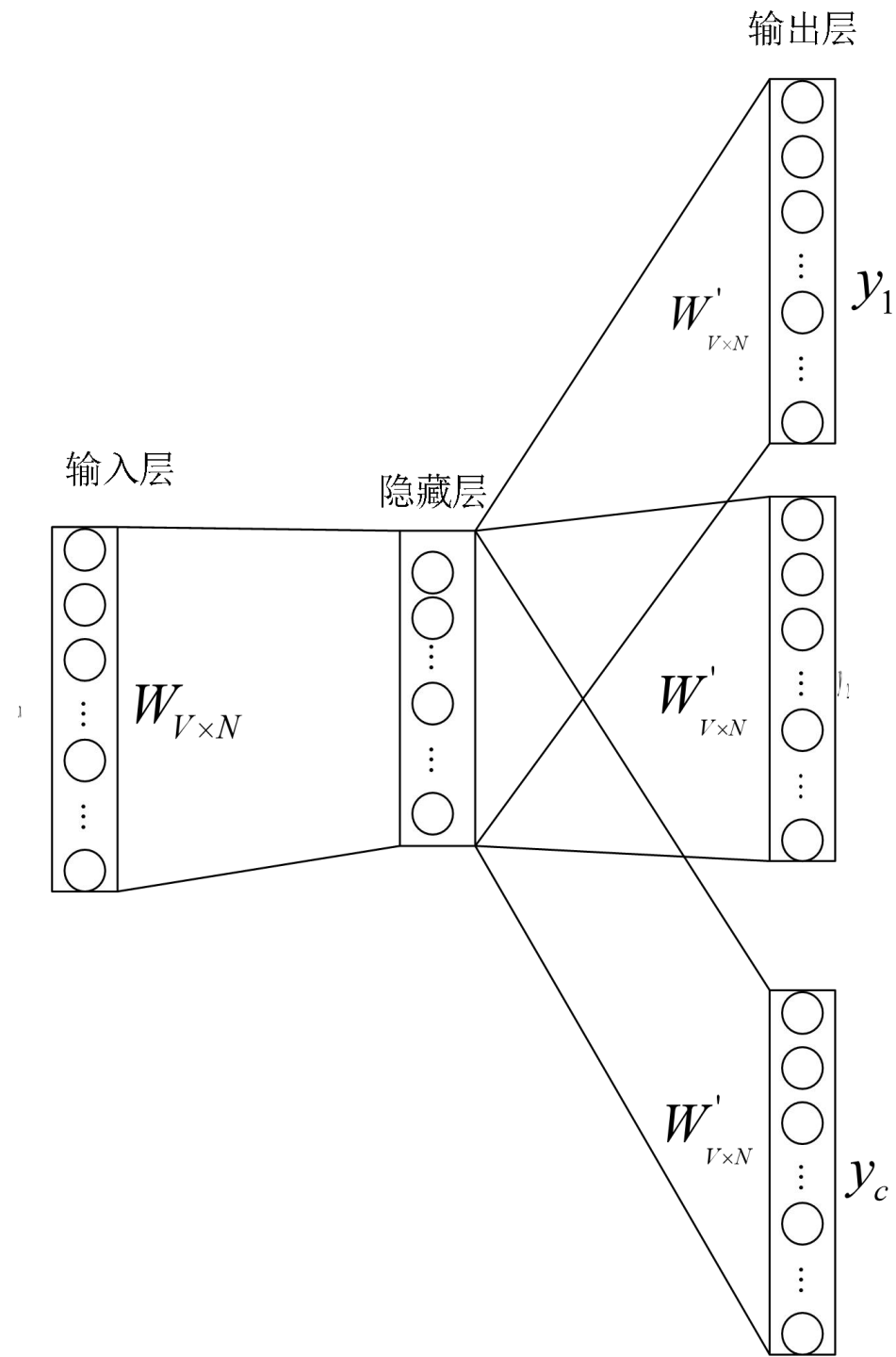
• (4)CBOW模型学习步骤

- 假设词向量空间维度为 V ，上下文词的个数为 C ，词汇表中的所有词都转化为one-hot向量，CBOW模型的学习步骤如下。
 - 初始化权重矩阵 W ($V \times N$ 矩阵， N 为人为设定的隐藏层单元的数量)，输入层的所有one-hot向量分别乘以共享的权重矩阵 W ，得到隐藏层的输入向量。
 - 隐藏层的输入向量求平均作为隐藏层的输出。
 - 隐藏层的输出向量乘以权重矩阵 W' ($N \times V$ 矩阵)，得到输出层的输入向量。
 - 输入向量通过激活函数处理得到输出层的概率分布。
 - 计算损失函数。
 - 更新权重矩阵。



Word2Vec模型

- 2. Skip-Gram模型
- Skip-Gram模型与CBOW模型相反，是根据目标词预测其上下文。其结构图如图所示。





Word2Vec模型

- (1)Skip-Gram模型的结构
- 假设词汇表中词汇量的大小为 V ，隐藏层的大小为 N ，相邻层的神经元是全连接的，Skip-Gram模型的结构组成如下。
 - 输入层含有 V 个神经元，输入是一个 V 维one-hot向量。
 - 输入层到隐藏层连接权重是一个 $V \times N$ 维的权重矩阵 W 。
 - 输出层含有 C 个单元，每个单元含有 V 个神经元，隐藏层到输出层每个单元连接权重共享一个 $N \times V$ 维权重矩阵 W' 。
 - 输出层每个单元使用softmax函数计算得到上下文的概率分布。





Word2Vec模型

• (2)Skip-Gram模型的数字形式

- 假设给定一个输入 x ，则隐藏层的输出为一个 N 维向量，表达式为 $h = x^T W$ 。
- 输出层有 $C \times V$ 个输出神经元，每个神经元节点的净输入则表示为 $u_{c,j} = v'_{W_j}{}^T \cdot h$ ，其中 $u_{c,j}$ 为输出层第 c 个单元的第 j 个神经元的净输入， v'_{W_j} 是矩阵 W' 的第 j 列向量。
- 由于每个输出单元共享相同的 W' ，所以每个单元的第 j 个神经元的净输入相同，即 $u_{c,j} = u_j$ 。
- 最终可得输出层第 c 个单元，第 j 个神经元的输出表达式如下。

$$y_{c,j} = p(w_{c,j} = w_{o,c} | w_I) = \frac{\exp(u_{c,j})}{\sum_{j'=1}^V \exp(u_{j'})}, j = 1, 2, \dots, V$$





Word2Vec模型

- (3)Skip-Gram模型的损失函数
- 模型训练的目标是，给定一个特定词，使得输出的 C 个单元为实际的 C 个上下文的概率最大，即最大化条件概率，其表达式如下。

$$p(w_{o,1}, w_{o,2}, \dots, w_{o,c} | w_I) = p(w_{o,1} | w_I) p(w_{o,2} | w_I) \cdots p(w_{o,c} | w_I)$$





Word2Vec模型

- Skip-Gram模型的损失函数定义式如下。其中， j_c^* 是第 c 个单元上的词在词汇表中的索引。

$$\begin{aligned} E &= -\log p(w_{o,1}, w_{o,2}, \dots, w_{o,c} | w_I) \downarrow \\ &= -\log p(w_{o,1} | w_I) p(w_{o,2} | w_I) \cdots p(w_{o,c} | w_I) \downarrow \\ &= -\log \prod_{c=1}^C \frac{\exp(u_{c,j_c^*})}{\sum_{j'=1}^V \exp(u_{j'})} \downarrow \\ &= \sum_{c=1}^C u_{c,j_c^*} + C \cdot \log \sum_{j'=1}^V \exp(u_{j'}) \end{aligned}$$





Doc2Vec模型

- Word2Vec模型获取一段文本的向量，是先对文本分词，提取文本的关键词，用Word2Vec获取这些关键词的词向量，然后计算这些关键词向量的平均值。
- 这种方法只保留了句子或文本中词的信息，丢失了文本中的主题信息。
- 为此，有研究者在Word2vec的基础上提出了文本向量化Doc2Vec模型。





Doc2Vec模型

- Doc2Vec模型与Word2Vec模型类似，只是在Word2Vec模型输入层增添了一个与词向量同维度的段落向量，可以将这个段落向量看作是另一个词向量。
- Doc2Vec存在如下两种模型。
 - 分布式记忆模型（Distributed Memory, DM），对应Word2Vec模型里的CBOW。
 - 分布式词袋模型（Distributed Bag of Words, DBOW），对应Word2Vec模型中的Skip-gram模型。





Doc2Vec模型

- 1. DM模型

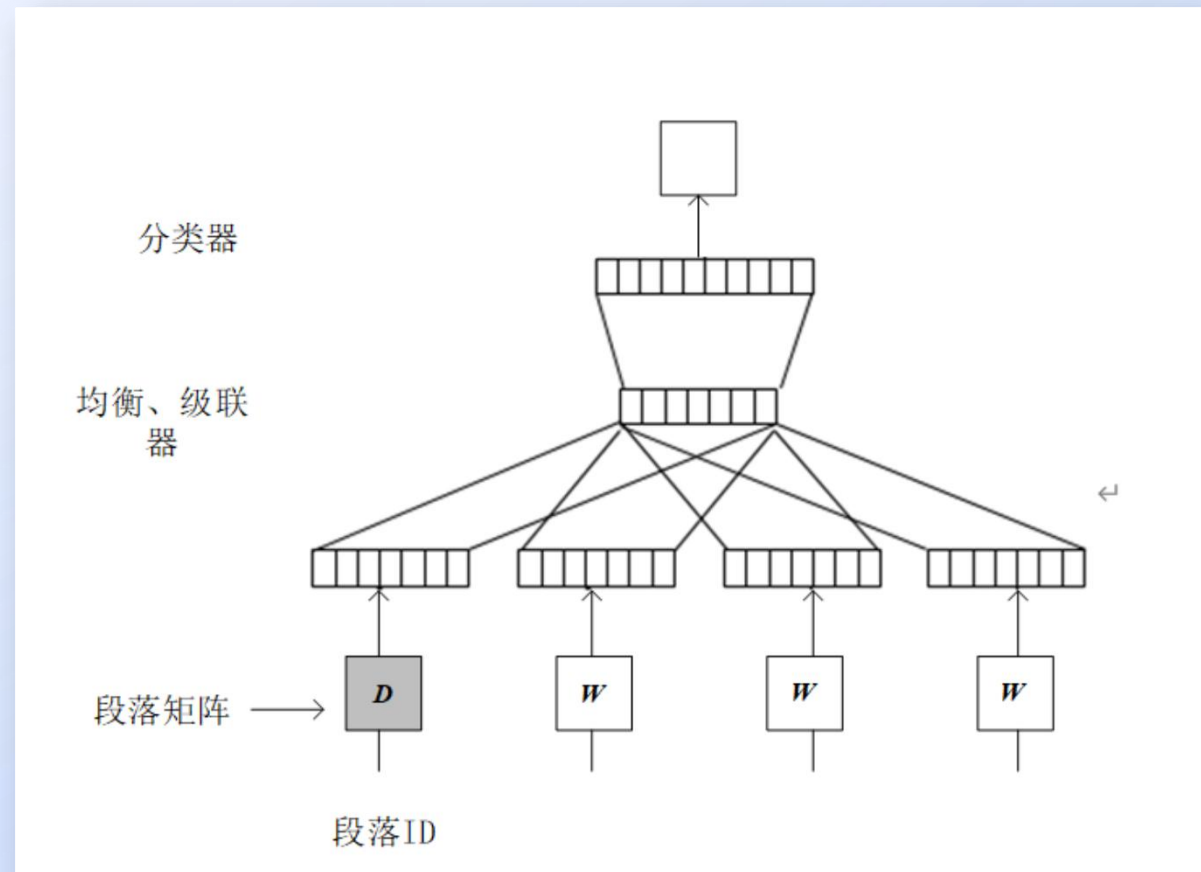
- DM模型与CBOW模型类似，在给定上下文的前提下，试图预测目标词出现的概率，只不过DM模型的输入不仅包括上下文而且还包括相应的段落。



Doc2Vec模型

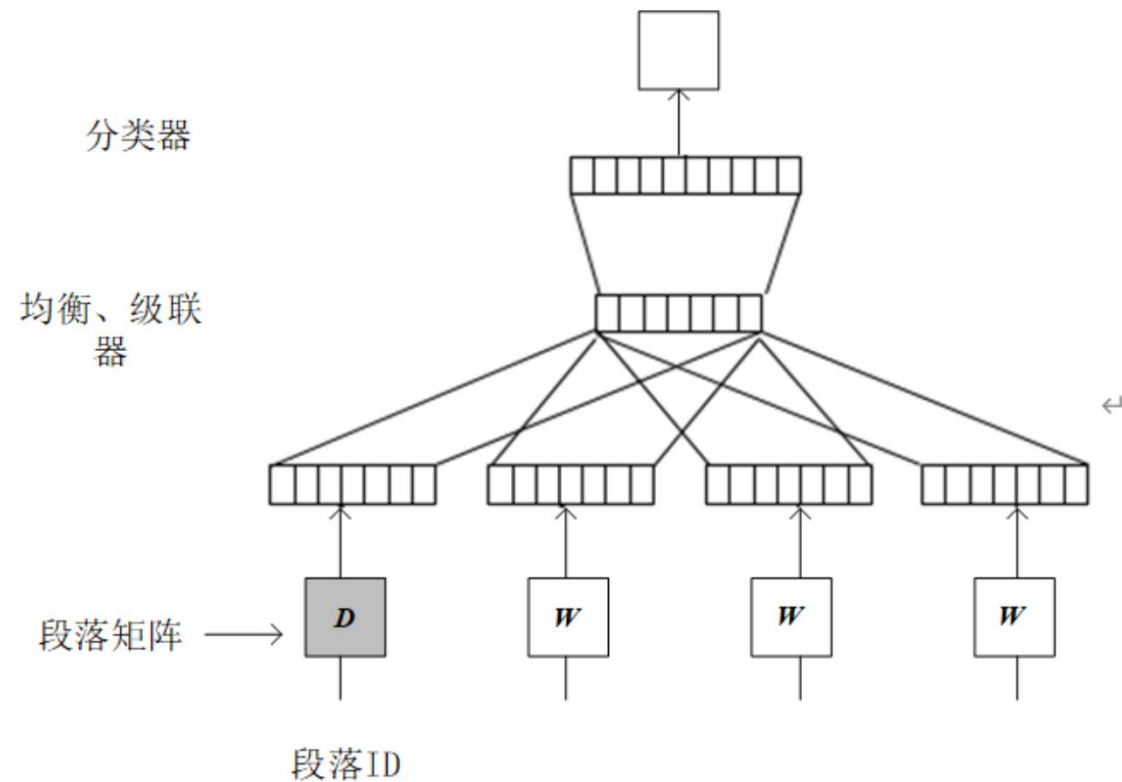
• 1. DM模型

- 假设词汇表中词汇量的大小为 V ，每个词都用one-hot向量表示，神经网络相邻层的神经元是全连接的，DM模型的网络结构如下。
 - 输入层含1个段落单元， $2n$ 个上下文单元，每个单元有 V 个神经元，用于输入 V 维one-hot向量。
 - 隐藏层的神经元个数为 h ，段落单元到隐藏层链接权重为 D 维矩阵，每个上下文单元到隐藏层连接权重值共享一个 V 维的权重矩阵。
 - 输出层含有 h 个神经元，隐藏层到输出层连接权重为 W 维权重矩阵。
 - 通过softmax函数计算输出层的神经元输出值。



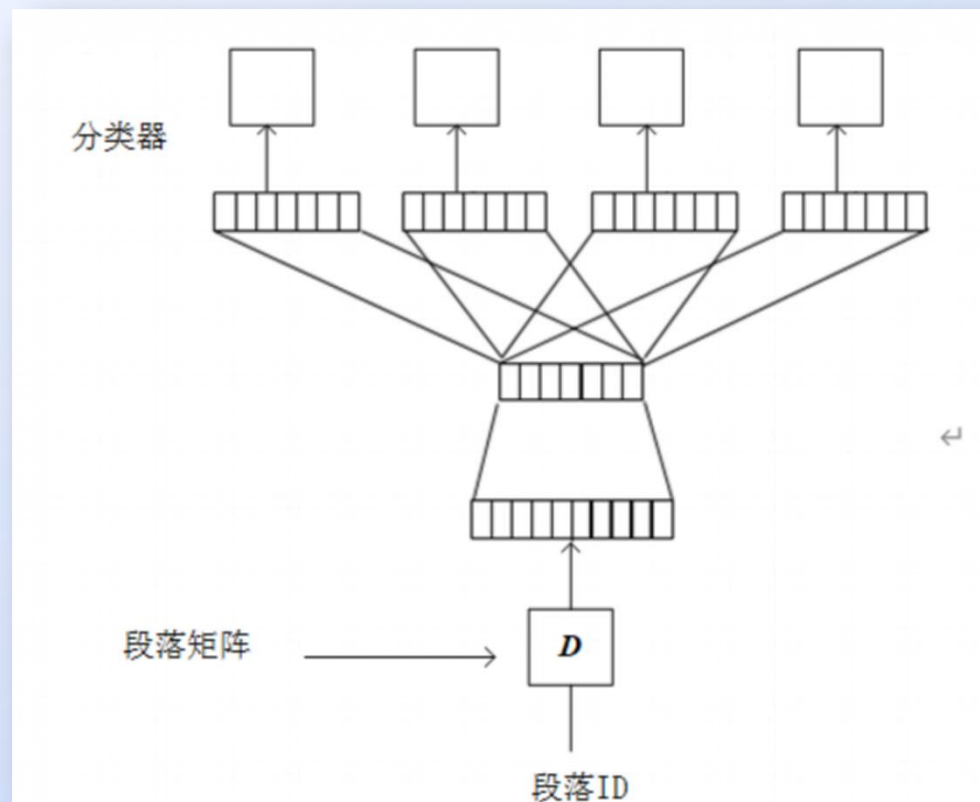
Doc2Vec模型

- DM模型增加了一个与词向量长度相等的段落向量即Paragraph ID，从输入到输出的计算过程如下，结构如图所示。
- Paragraph ID通过矩阵 映射成段落向量。段落向量和词向量的维数虽然一样，但是代表了两个不同的向量空间。每个段落或句子被映射到向量空间中，可以用矩阵的一列表示。
- 上下文通过矩阵 映射到向量空间，用矩阵 的一列表示。
- 将段落向量和词向量求平均或按顺序拼接后输入Softmax层。



Doc2Vec模型

- DBOW模型
- 与Skip-gram模型只给定一个词语预测上下文概率分布类似，DBOW模型输入只有段落向量，DBOW模型通过一个段落向量预测段落中随机词的概率分布。其结构如图所示。





Doc2Vec模型

• 2. DBOW模型

- DBOW模型的训练方法忽略输入的上下文，让模型去预测段落中的随机一个词，在每次迭代的时候，从文本中采样得到一个窗口，再从这个窗口中随机采样一个词作为预测任务并让模型去预测，输入就是段落向量。
- Doc2Vec模型主要包括以下两个步骤。
 - 训练模型。在已知的训练数据中得到词向量、各参数项和段落向量或句子向量。
 - 推断过程。对于新的段落，需要得到它的向量表达。





Doc2Vec模型

- 2. DBOW模型

- DM模型与CBOW模型相对应，可以根据上下文词向量和段落向量预测目标词的概率分布。
DBOW模型与Skip-gram模型对应，只输入段落向量，预测从段落中随机抽取的词组概率分布。总而言之，Doc2vec是Word2Vec的升级，Doc2vec不仅提取文本的语义信息，而且提取了文本的语序信息。





分布式表示方法的特点

- **优点：**能够有效地捕捉单词之间的语义和句法关系，将具有相似语义的单词映射到向量空间中相近的位置，有助于理解文本的语义内容。同时，分布式表示将高维的文本数据转换为低维向量，大大减少了数据的存储空间和计算量，提高了模型的训练和推理效率。
- **缺点：**训练过程通常需要大量的文本数据和较高的计算资源，训练时间较长。而且，虽然分布式表示能够捕捉到一定的语义信息，但对于一些复杂的语义关系和上下文理解，可能还存在一定的局限性，并且模型的可解释性相对较差，不如离散表示直观。



目录 CONTENTS



01

文本向量化简介

02

文本离散表示

03

分布式表示

04

论文相似度计算



Word2Vec词向量的训练

• 1. 中文语料预处理

- 训练词向量需要有一个大量的语料库，本节使用维基百科的中文网页数据做为训练语料库。
- 预处理主要包含以下三个步骤。
 - 将维基百科提供的xml格式的语料文件读入后存储为txt格式。
 - 将繁体字转化为简体字。
 - 利用jieba库对语料库中的句子进行分词。

• 2. 词向量训练

- 当中文语料预处理完成后，使用gensim库训练词向量。
- 由于维基百科的语料数据较多，所以对它进行数据预处理和训练时需要等待较长的时间。在代码中设置判断语句，如果训练好的模型存在则不需要再训练。





Doc2Vec段落向量的训练

• 1. 数据预处理

- 定义一个类包含繁体字和简体字的转换操作和利用jieba库进行分词操作。
- gensim库里的Doc2vec提供的LabeledSentence函数可以将文档标签（如类别）加入到文档向量的训练中，因此Doc2vec在训练时能够采用tags信息更好地辅助训练，相对于Word2Vec模型，这里的多了一个tags属性。

• 2. 段落向量训练

- 当数据预处理完成后，使用gensim库训练段落向量。
- Doc2Vec函数训练段落向量，第一个参数documents表示预处理后用于训练的语料；dm=0表示使用DBOW模型训练段落向量，dm=1则表示使用DM模型训练段落向量；参数size表示段落向量的维度，参数window、min_count、workers的含义与Word2Vec函数相同。





计算文本的相似度

- 1. Word2Vec计算文本相似度
 - 使用Word2Vec模型计算文本相似度，包括以下3个步骤。
 - 抽取文本中的关键词。采用jieba库里的tfidf函数提取关键词。
 - 使用Word2Vec模型将关键词向量化，并将所有关键词的词向量相加，得到文本的向量化表示。
- 利用文本的向量化表示计算文本的相似度
 - 用自定义similarity函数计算向量间余弦值，该余弦值用于表示两个向量的相似度，最终得出两份文本的相似度为92.28%。

利用gensim.models.wv.similarity函数计算相似度





计算文本的相似度

- 2.Doc2Vec计算文本相似度
 - 使用Doc2Vec模型计算文本相似度，主要包括以下3个步骤。
 - 数据预处理。
 - 文档向量化。
 - 计算文本相似度。
- similarity函数计算两个向量的相似度。运行代码后可得出两份文本的相似度为81.72%





计算文本的相似度

• 3. 两种相似度计算方法分析

- 两种方法计算文本相似度的结果显示，利用Word2Vec方法计算的相似度为92.28%，高于Doc2Vec方法计算的81.72%。
- 通常情况下Doc2Vec的效果可能会更加准确，这是因为Doc2Vec较之Word2Vec，除了利用词语的语义信息外，还综合了上下文的语序信息，而Word2Vec在训练时则会丢失语序信息。
- 通过查看Word2Vec提取的两份文本关键词发现它们的多数词语相同，因此推测原因可能由于是这两份文本本身较为相似，另外Word2Vec方法中的关键词提取算法准确率达到较高的水平，保留了许多关键信息。





小结

- 本章主要介绍了文本向量化的基本概念和两种文本的表示方法。
- 文本的表示方法介绍了文本的离散表示和分布式表示方法。
- 离散表示介绍了one-hot、BOW模型和TF-IDF3种表示方法。
- 分布式表示介绍了Word2Vec模型和Doc2Vec模型各自的两个模型。



THANK YOU

